

非对称稠密矩阵的延迟主元LU方法

Step1 分解Fully Summed块

Step2 更新波前矩阵

Step3 传递贡献块给父节点

Step3.1 子节点传出的内容

Step3.2 父节点建立新的索引集

Step3.3 建立局部映射并累加数值

Step3.4 组装后的分区与继续传递

对称稠密矩阵的Bunch-Kaufman方法

对称Step1: 分解Fully-Summed块

Step1.1 Bunch-Kaufman主元搜索

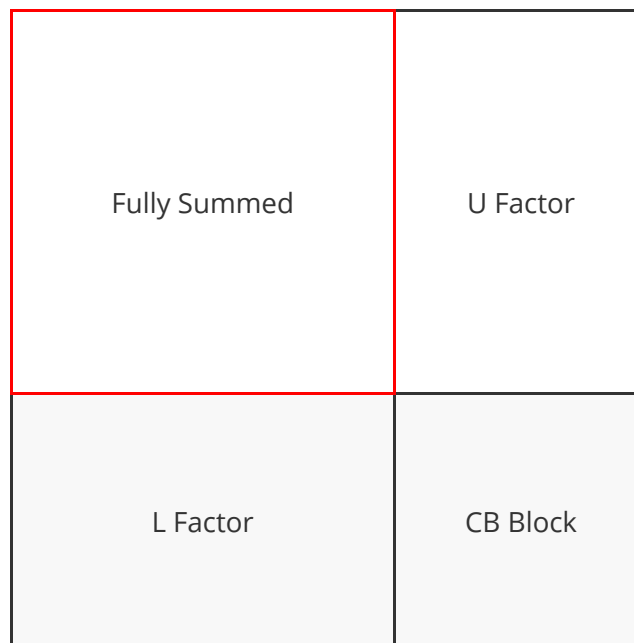
Step1.2 对称置换

Step1.3 检查 2×2 主元的稳定性

Step1.4 计算 L 并更新活动子块

非对称稠密矩阵的延迟主元LU方法

- 在超节点方法中，我们会针对单个超节点的主元块将其作为一个稠密矩阵进行LU分解，按照列主元的方法，我们针对每一列进行选主元，然后更新。若是出现某个列待分解的部分全0，那么我们就选不出列主元，此时Lapack中dgetrf函数的做法是先跳过此列，继续进行下一列的分解，最后报错，这样的做法显然不符合我们最终求解的需要。
- 一个子主元块无法分解，不代表全部的矩阵无法分解。我们可以把无法分解的部分传递给父节点再尝试分解。

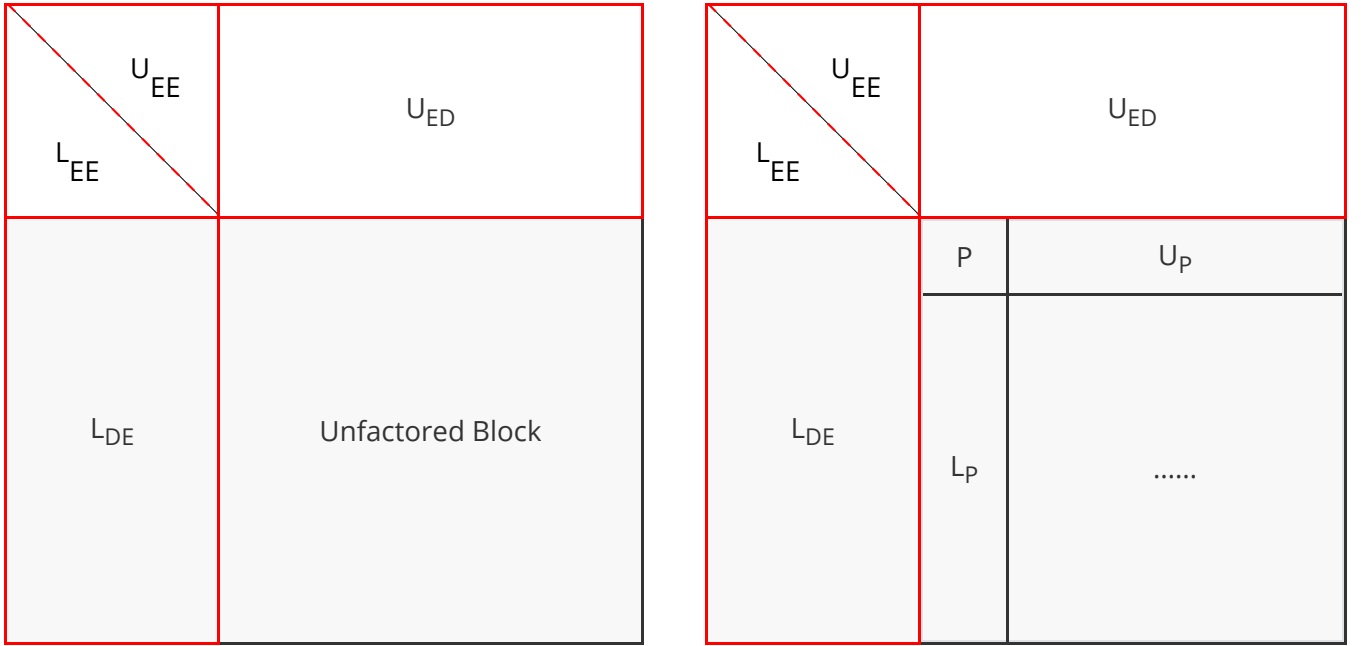


- 我们需要处理如上图所示的波前矩阵：
 - Fully-Summed代表着已经装配好可以进行分解的矩阵，即为超节点的对角块；
 - L/U Factor是我们需要根据Fully-Summed部分的分解结果更新的块，作为最终的结果呈现；
 - CB Block是子节点对于父节点的贡献块，需要在更新之后传递给父节点。
- 注意到对角块之外的行列来自于和对角块中行列耦合的行列。

- 在此方法中，我们仍采用right-looking的列主元方法+KJL更新方法。

Step1 分解Fully Summed块

- 我们首先单独考虑对Fully Summed块的分解。
- 红色部分的块表示已经分解好的部分，接下来需要对Unfactored Block进行分解，一个比较自然的思路是：
 - 首先进行选列主元，如果能选到合适的列主元，则进行行交换P，将其作为主元计算；
 - 若不能选取到合适的列主元，那么我们就针对全局选取主元，此时需要做行列交换，将右下角的全局最大主元置换到P的位置，然后根据P的值计算 L_p ，然后更新 U_p 和右下角块。



- 设第 k 步尚未分解的活动子块为：

$$S^{(k)} = \left[s_{ij}^{(k)} \right]_{i,j=k}^f, \quad (1)$$

- 其中 f 为Fully Summed块的阶数， $\tau \in (0, 1)$ 为主元接受阈值。

列主元：只搜索活动子块的第 k 列，候选主元所在行为

$$p_k = \operatorname{argmax}_{k \leq i \leq f} \left| s_{ik}^{(k)} \right|, \quad \alpha_k = \left| s_{p_k k}^{(k)} \right|. \quad (2)$$

- 记活动子块中的最大元素为：

$$\beta_k = \|S^{(k)}\|_{\max} = \max_{k \leq i, j \leq f} \left| s_{ij}^{(k)} \right|. \quad (3)$$

当 $\alpha_k \geq \tau \beta_k$ 时，认为列主元足够稳定。用行置换矩阵 P_k 交换第 k 行和第 p_k 行：

$$\tilde{S}^{(k)} = P_k S^{(k)}, \quad d_k = \tilde{s}_{kk}^{(k)} = s_{p_k k}^{(k)}. \quad (4)$$

全主元：若 $\alpha_k < \tau\beta_k$ ，则在整个活动子块中搜索最大元素：

$$(p_k, q_k) = \operatorname{argmax}_{k \leq i, j \leq f} |s_{ij}^{(k)}|. \quad (5)$$

分别用 P_k 和 Q_k 将第 p_k 行、第 q_k 列交换到第 k 行、第 k 列：

$$\tilde{S}^{(k)} = P_k S^{(k)} Q_k, \quad d_k = \tilde{s}_{kk}^{(k)} = s_{p_k q_k}^{(k)}, \quad |d_k| = \beta_k. \quad (6)$$

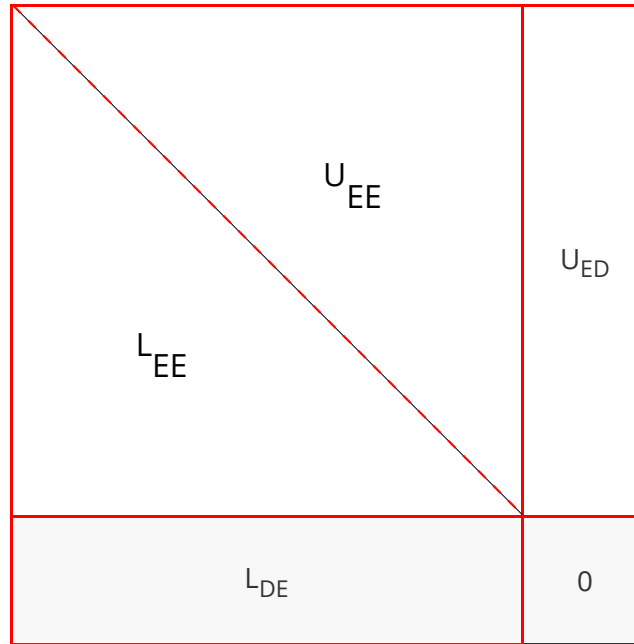
因此，只使用列主元时最终满足 $PA = LU$ ；使用过全主元时则应记录行、列两种置换，最终满足 $PAQ = LU$ 。

更新：主元 d_k 确定后，对主元行、主元列和右下角剩余子块进行right-looking更新：

$$(L_p)_i = \frac{\tilde{s}_{ik}^{(k)}}{d_k}, \quad k < i \leq f, \quad (U_p)_j = \tilde{s}_{kj}^{(k)}, \quad k < j \leq f, \quad (7)$$

$$s_{ij}^{(k+1)} = \tilde{s}_{ij}^{(k)} - (L_p)_i (U_p)_j = \tilde{s}_{ij}^{(k)} - \frac{\tilde{s}_{ik}^{(k)} \tilde{s}_{kj}^{(k)}}{d_k}, \quad k < i, j \leq f. \quad (8)$$

- 若 $\beta_k = 0$ ，则 $S^{(k)}$ 为零块，列主元和全主元都不存在，在数值意义下不可继续分解，需要将未分解变量延迟到父节点。
- 完成上述步骤后 P 、 U_p 、 L_p 会被归于已分解部分，剩下的作为Unfactored Block进行下一步迭代。



- 实际上若是完全采用上面的做法，那么算法将会消耗掉巨量的时间在搜索全部元与更新上，于是我们提出以下两种优化方法：

列延迟：仔细思考全主元算法，我们发现：利用行列变换将全局最大值替换到我们正在进行分解的第 k 个主元，本质上是全局最大值那一列通过列交换交换到了第 k 列，然后进行了一次列主元分解，而本身的第 k 列因为已经选不出全局主元，故第 k 列一定是全0的，那么在之后的分解过程中，全零列既不会被选出全主元来，也不会被更新。原因是被放到右边的列在当前活动部分全为零： $A(k:m-1, j) = 0$ 。之后的LU更新为 $A(k+1:m-1, j) \leftarrow A(k+1:m-1, j) - L(:, k)U(k, j)$ 。因为 $U(k, j) = 0$ ，所以更新项也是零。所以选全主元的步骤可以被替换为以下列延迟步骤：

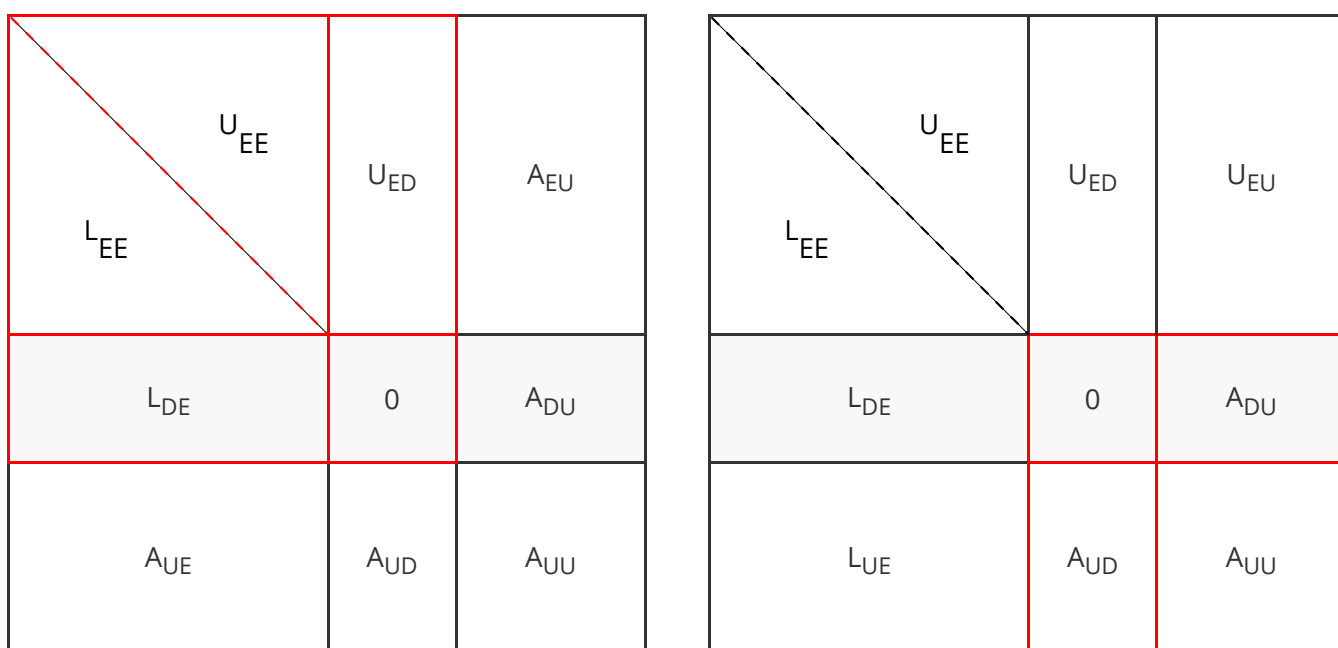
- 若 $\alpha_k < \tau\beta_k$ ，则我们将该列直接移动到矩阵的 Unfactored Block 的尾部去，并记录这个交换的步骤。
- 为了保证之后的列进行延迟被移动到尾部的时候不会再次将本来就被存放在 Unfactored Block 的尾部的零列给置换到前方去，从而导致死循环，我们可以通过以下算法避免问题：
 - 从右向左找第一列可用列，找到后立即停止，不比较其他列。
 - 在该列内部使用列主元法，并将其和当前的零列替换位置。‘
 - 最后如果我们搜索到当前列都找不到可用列，那么说明所有的 Unfactored Block 已经全零，剩下的所有主元全部延迟。

更细粒度的自适应KJI-SAXPY方法优化：在Lapack的dgetrf中，采用到了类似的优化方法（之后的step2也是这个思路）：我们如果每次选主元后立刻更新所有项目，那么显然是会很慢的，我们可以借助blas的三角求解和矩阵乘法针对矩阵块进行快速更新，这就需要在分解一部分主元之后一起更新。

- demo中选取panel长度为64，然后在这个panel中，我们选列主元（注意不是在64*64的panel块中选，而是在当前列的所有活动列中选取，所以说panel其实是一个长方形矩阵）后只更新当前长方形panel中的数，只需要保证选主元的时候当前列已经被更新就行了，当选完一个panel之后，我们在更新剩下的部分，然后再选取新的panel进行更新
- 但是我们的流程中会遇到延迟主元的情况，此时会对当前panel产生影响，为了保证数值分解的正确性，我们让算法在遇到延迟主元的时候立刻停止当前panel的分解，并按照已经分解的部分将矩阵的剩余部分全部更新一次，然后再从被换过来的非零列开始新一个panel的分解。这样既做到了利用blas对整个分解过程加速，也保证了数值分解的正确性。
- 经过测试后，采用列延迟的方法大概可以将整个计算过程时间缩短90%，而采用更细粒度的自适应KJI-SAXPY方法优化后速度还能提升一倍，整体优化后的算力能力在恶意设计过的算例上的时间已经略快于Lapack。

Step2 更新波前矩阵

- 此时波前矩阵的Fully-Summed的部分已经被分成了5个部分，我们根据已经分解完毕的情况对其他部分进行更新：



Step2.1 更新 A_{UE} 、 A_{EU} 得到 L_{UE} 、 U_{EU} ：

- 利用三角求解计算：

$$U_{EU} = L_{EE}^{-1} A_{EU} \quad L_{UE} = A_{UE} U_{EE}^{-1} \quad (9)$$

Step2.2 更新 A_{DU} 、 A_{UD} ：

$$A_{DU} \leftarrow A_{DU} - L_{DE} U_{EU} \quad (10)$$

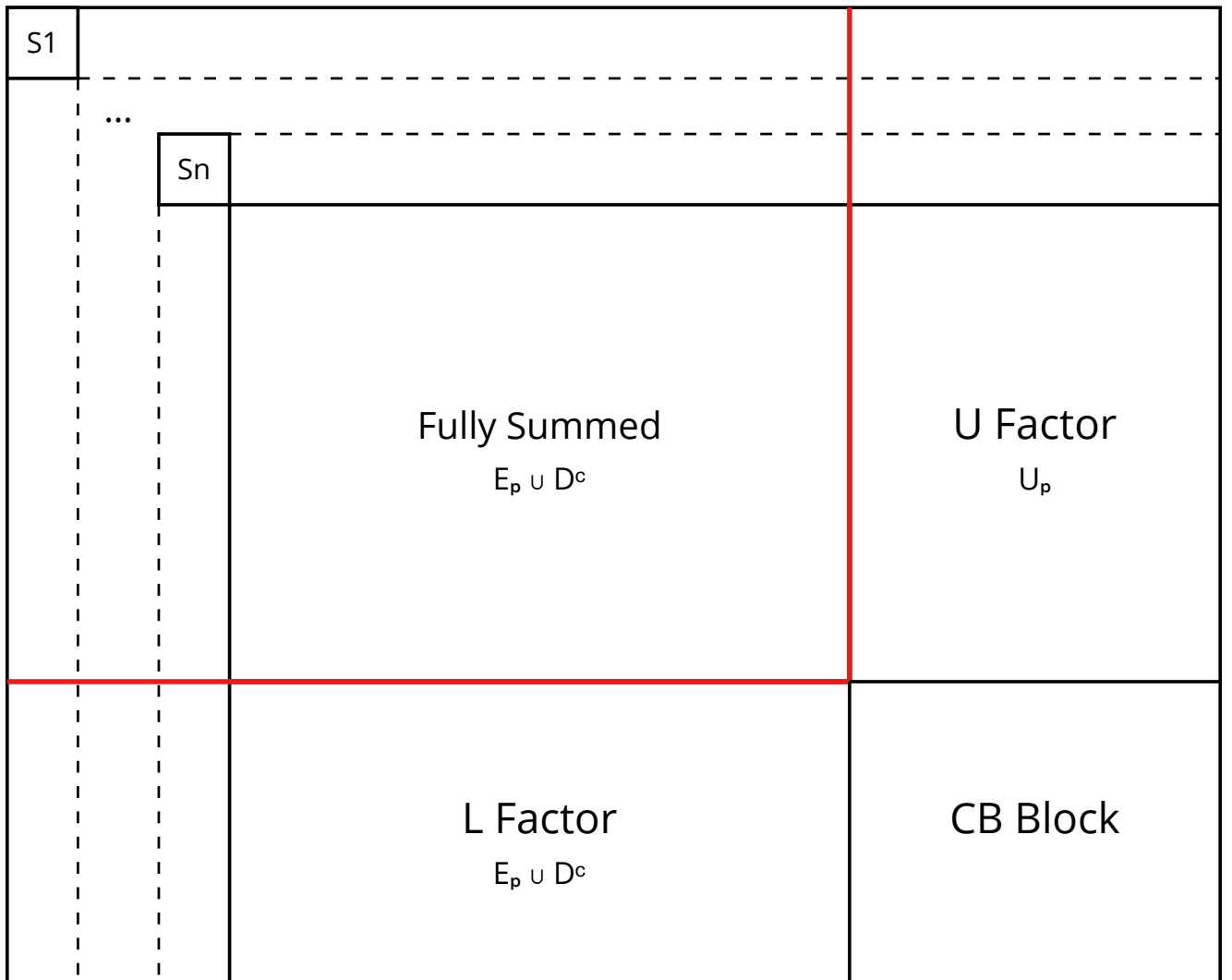
$$A_{UD} \leftarrow A_{UD} - L_{UE} U_{ED}$$

Step2.3 更新 A_{UU} ：

$$A_{UU} \leftarrow A_{UU} - L_{UE} U_{EU} \quad (11)$$

Step3 传递贡献块给父节点

- 在Step2中我们更新了所有的部分，并将右下的四个子矩阵组成扩展贡献块传递给父节点组装。
- 这里的“组装”不是把整个稠密块按几何位置直接拼到父矩阵上，而是根据贡献块的**全局行列编号**做 `extend-add`（扩展—累加）。
- 普通更新变量与父节点已有索引匹配后累加，子节点未消去的延迟变量则动态扩充父节点的候选主元区域。



Step3.1 子节点传出的内容

- 设子节点已消去的变量为 E_c ，未能消去的延迟变量为 D_c ，原来的普通更新变量为 U_c 。子节点传出的扩展贡献块为

$$C_c = \begin{bmatrix} S_{DD} & S_{DU} \\ S_{UD} & S_{UU} \end{bmatrix}, \quad I_c^{\text{out}} = D_c \cup U_c. \quad (12)$$

- 除了数值 C_c 以外，还必须同时传递其全局行、列索引数组。对于非对称LU，行索引和列索引应分开保存：

$$R_c^{\text{out}} = D_c^r \cup U_c^r, \quad C_c^{\text{out}} = D_c^c \cup U_c^c. \quad (13)$$

- 数值块中的第 (i, j) 个元素，必须与 $R_c^{\text{out}}(i)$ 和 $C_c^{\text{out}}(j)$ 两个全局编号绑定，不能只传递一个没有索引的稠密数组。

Step3.2 父节点建立新的索引集

- 设父节点原来的候选主元集为 E_p ，更新集为 U_p 。正确的符号分析通常应保证子节点的普通更新变量已在父波前中，即：

$$U_c \subseteq E_p \cup U_p. \quad (14)$$

- “普通更新变量”是相对子节点而言的。它们进入父节点后要根据符号分析确定的消去归属再次分类：
 - 其中 $U_c \cap E_p$ 在子节点只是更新变量，但到达父节点后已经Fully Summed，可在父节点参与选主元；
 - $U_c \cap U_p$ 则仍然留在父节点的更新区。

$$U_c = (U_c \cap E_p) \cup (U_c \cap U_p). \quad (15)$$

- 延迟变量 D_c 是数值分解的过程中动态产生的部分，因此父节点需要将其加入新的候选主元集：

$$E_p^{\text{new}} = E_p \cup D_c, \quad I_p^{\text{new}} = E_p^{\text{new}} \cup U_p. \quad (16)$$

Step3.3 建立局部映射并累加数值

- 父节点为每个全局行列编号建立到局部存储位置的映射：

$$\text{map}_r(g) = g \text{ 在父波前行索引中的局部位置}, \quad (17)$$

$$\text{map}_c(g) = g \text{ 在父波前列索引中的局部位置}. \quad (18)$$

- 若扩展后出现新的行或列，则先扩展父波前的存储空间，并将新位置初始化为零。然后对子贡献块执行散射累加：

$$F_p(\text{map}_r(R_c^{\text{out}}(i)), \text{map}_c(C_c^{\text{out}}(j))) += C_c(i, j). \quad (19)$$

- 父矩阵中原来没有数值的位置按零处理，累加子节点贡献后就成为新的fill-in。如果多个子节点对同一个全局位置有贡献，则必须将它们全部累加，而不是相互覆盖。

Step3.4 组装后的分区与继续传递

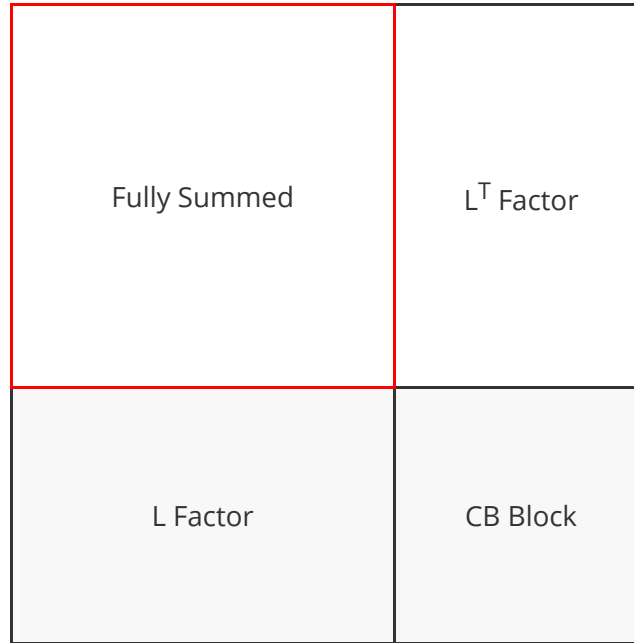
- 完成所有子贡献块的 `extend-add` 之后，父节点按下列规则进行操作：
 - $E_p \cup D_c$ ：放入Fully Summed候选主元区，在父节点重新进行主元检验；
 - 重复step1、step2在父节点成功消去的延迟变量进入 L/U 因子；
 - 仍无法通过主元检验的变量再次进入父节点的扩展贡献块，继续向更高层节点传递。

对称稠密矩阵的Bunch-Kaufman方法

- 如果整个波前矩阵是对称的，则Fully-Summed块也是对称的，此时可采用Bunch-Kaufman方法进行带对称主元选取的 LDL^T 分解：

$$P^T A P = L D L^T. \quad (20)$$

- 其中 P 是用于同时置换行、列的置换矩阵， L 为单位下三角矩阵， D 为由 1×1 和 2×2 主元块组成的块对角矩阵。与非对称LU不同，对称分解中不能只交换行而不交换列；交换第 i 、 j 行时，必须同时交换第 i 、 j 列。



对称Step1：分解Fully-Summed块

- 设第 k 步尚未分解的活动对称子块为

$$S^{(k)} = \left[s_{ij}^{(k)} \right]_{i,j=k}^f, \quad S^{(k)} = \left(S^{(k)} \right)^T, \quad (21)$$

- 其中 f 是当前仍可参与主元选取的Fully-Summed变量个数。Bunch-Kaufman方法在每一步选择一个 1×1 或 2×2 对角主元块，并保持后续更新仍然对称。

Step1.1 Bunch-Kaufman主元搜索

定义Bunch-Kaufman阈值

$$\gamma = \frac{1 + \sqrt{17}}{8} \approx 0.640388. \quad (22)$$

- 首先检查第 k 列。记当前对角元大小为

$$a_k = |s_{kk}^{(k)}|, \quad (23)$$

- 第 k 列对角线以下的最大元素所在行为

$$p = \operatorname{argmax}_{k < i \leq f} |s_{ik}^{(k)}|, \quad \lambda_k = \max_{k < i \leq f} |s_{ik}^{(k)}| = |s_{pk}^{(k)}|. \quad (24)$$

- 若该列不为数值零，则按以下规则选择主元。
 1. 若 $a_k \geq \gamma \lambda_k$ ，则直接采用 $s_{kk}^{(k)}$ 作为 1×1 主元，不需要交换。
 2. 若 $a_k < \gamma \lambda_k$ ，则检查候选行 p 。定义该行除对角元之外的最大元素：

$$\sigma_p = \max_{\substack{k \leq j \leq f \\ j \neq p}} |s_{pj}^{(k)}|. \quad (25)$$

因为 $|s_{pk}^{(k)}| = \lambda_k$ ，所以当 $\lambda_k > 0$ 时有 $\sigma_p \geq \lambda_k > 0$ 。接下来依次判断：

- 若 $a_k \geq \gamma \frac{\lambda_k^2}{\sigma_p}$ ，仍使用 $s_{kk}^{(k)}$ 作为 1×1 主元；
- 否则，若 $|s_{pp}^{(k)}| \geq \gamma \sigma_p$ ，则同时交换第 k 、 p 行和第 k 、 p 列，将 $s_{pp}^{(k)}$ 移到 (k, k) 位置，并将其作为 1×1 主元；
- 若上述两个条件都不满足，则选择由第 k 、 p 个变量组成的 2×2 主元块。通过对称交换将变量 p 移动到 $k+1$ 位置，得到：

$$D_k = \begin{bmatrix} \tilde{s}_{kk}^{(k)} & \tilde{s}_{k,k+1}^{(k)} \\ \tilde{s}_{k+1,k}^{(k)} & \tilde{s}_{k+1,k+1}^{(k)} \end{bmatrix}. \quad (26)$$

Step1.2 对称置换

- 对于 1×1 主元 $s_{pp}^{(k)}$ ，用置换矩阵 P_k 同时交换第 k 、 p 行和第 k 、 p 列：

$$\tilde{S}^{(k)} = P_k^T S^{(k)} P_k, \quad D_k = \begin{bmatrix} \tilde{s}_{kk}^{(k)} \end{bmatrix}. \quad (27)$$

- 对于由 k 、 p 组成的 2×2 主元，用 P_k 将这两个变量放到活动子块的前两个位置：

$$\tilde{S}^{(k)} = P_k^T S^{(k)} P_k = \begin{bmatrix} D_k & B_k^T \\ B_k & C_k \end{bmatrix}, \quad D_k \in \mathbb{R}^{2 \times 2}. \quad (29)$$

- 置换必须同时作用于整个波前矩阵中对应的行和列，并同步更新全局变量编号。多步置换累积后，最终得到

$$P = P_1 P_2 \cdots P_t, \quad P^T F P = L D L^T. \quad (30)$$

Step1.3 检查 2×2 主元的稳定性

- 设

$$D_k = \begin{bmatrix} a & b \\ b & c \end{bmatrix}, \quad \Delta_k = \det(D_k) = ac - b^2. \quad (31)$$

当 $\Delta_k \neq 0$ 时,

$$D_k^{-1} = \frac{1}{\Delta_k} \begin{bmatrix} c & -b \\ -b & a \end{bmatrix}. \quad (32)$$

- Bunch-Kaufman判据能够控制消元乘子的增长，但计算中仍应避免直接使用数值上近奇异的 2×2 块，可以进行额外检查：

$$\text{rcond}(D_k) = \frac{1}{\|D_k\| \|D_k^{-1}\|} \geq \tau_D. \quad (33)$$

若不计算条件数，也可以使用下面的行列式尺度判据做快速筛选（注意到）：

$$|\Delta_k| > \varepsilon_{\text{piv}} \|D_k\|^2. \quad (34)$$

若 D_k 不满足稳定性要求，则不能直接求 D_k^{-1} ，应将相关变量延迟到父节点，在更大的Fully-Summed候选集中重新选取主元。

Step1.4 计算 L 并更新活动子块

- 设本步主元阶数为 $r \in \{1, 2\}$ 。完成对称置换后，将子块写成

$$\tilde{S}^{(k)} = \begin{bmatrix} D_k & B_k^T \\ B_k & C_k \end{bmatrix}, \quad D_k \in \mathbb{R}^{r \times r}. \quad (35)$$

- 当前块列的消元乘子为

$$L_k = B_k D_k^{-1}, \quad (36)$$

右下角活动子块通过对称Schur补更新：

$$S^{(k+r)} = C_k - L_k D_k L_k^T = C_k - B_k D_k^{-1} B_k^T. \quad (37)$$

- 对于 1×1 主元 $D_k = [d_k]$ ，上述公式退化为

$$L_k = \frac{B_k}{d_k}, \quad S^{(k+1)} = C_k - \frac{B_k B_k^T}{d_k}. \quad (38)$$

- 对于 2×2 主元，使用

$$L_k = \frac{1}{\Delta_k} B_k \begin{bmatrix} c & -b \\ -b & a \end{bmatrix}, \quad (39)$$

然后执行秩2对称更新

$$S^{(k+2)} = C_k - L_k D_k L_k^T. \quad (40)$$

- 每成功消去一个 1×1 主元令 $k \leftarrow k + 1$ ；每成功消去一个 2×2 主元令 $k \leftarrow k + 2$ 。未通过主元检验的变量不进入 D ，而是移动到延迟区；若当前节点剩余的 Fully-Summed 变量都无法形成稳定主元，则停止本节点的数值消元。
- 延迟操作：如果存在找不到列主元的情况（列全0）则将其移到最后一个主元去准备做延迟。

块分解优化：和非对称情况一样，我们尝试用块分解的方法来进行优化。

1	[0, panel_begin)	以前面板已经消去
2	[panel_begin, k)	当前面板已经接受的主元
3	[k, active_end)	仍可参与主元选择的活跃节点
4	[active_end, n)	已经延迟的节点

设当前面板从 $b = \text{panel_begin}$ 开始。以前面板产生的 Schur 补已经全部写回 A ，所以在面板刚开始时：

- $A(b:n-1, b:n-1)$ 就是最新的尾矩阵

假设当前面板已经接受了 t 列： $L_P = L(:, b:b+t-1)$ ，对应的 1×1 和 2×2 主元组成： D_P 。

- 定义工作矩阵： $W_P = L_P D_P$ 。那么真正的当前 Schur 补是： $S = A_{\text{stored}} - L_P D_P L_P^T = A_{\text{stored}} - L_P W_P^T$ 。
- 当前准备处理第 k 列。因为 $A(:, k)$ 还没有包含本面板前面 t 列主元的更新，所以需要计算对当前列的更新之后才能继续选主元
- 按照 bunch-kaufman 方法，我们首先判定当前列上本来的对角元是否符合阈值要求，若是满足这标记作为主元
 - 若是不满足我们需要在接下来的判定中运用到当前列的最大值所在行的数据，此时需要更新此行
- 当 1×1 ， 2×2 主元都不满足要求，即需要延迟主元，我们维护一个 active_end 值记录尾部被延迟的行/列坐标，在每次需要延迟的时候利用这值向左查找找到下一个非零列，之后交换其与被延迟主元即可。
- 此优化要求在每个 panel 的分解过程中维护一个工作数组 W_P ：
 - 理论上可以只保存 L_P, D_P 。但每次需要当前列时，都要先计算： $D_P L_P(k, :)^T$ ，再计算： $L_P (D_P L_P(k, :)^T)$ 。而 D_P 中混合了 1×1 和 2×2 块，处理起来更复杂。
 - 预先保存： $W_P = L_P D_P$ 以后，当前列直接计算： $L_P W_P(k, :)^T$ 。只需要一次 `dgemv`，并且不必在每次更新时重新遍历 $1 \times 1 / 2 \times 2$ 的 D 块结构。
 - W_P 的大小为： $\text{panel_size} * \text{active_size}$ ，默认的设置下最大值为 $64 * 8192$ ，双精度存储下约耗费 4MiB。
 - 注意到：工作矩阵也需要随着矩阵在选主元的时候做的行列交换一起交换。